

Figure 1: Abstract view of conventional networking versus SDN

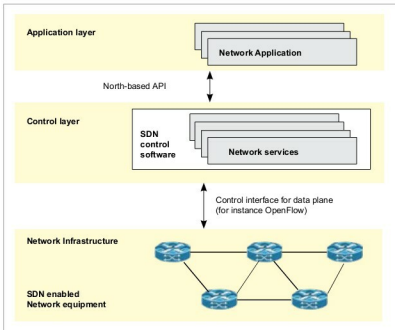


Figure 2: SDN or software defined networking

Advantages of SDN over traditional networking

SDN has many advantages over traditional networking:

- Due to the introduction of some automation in the process of networking through SDN, scalability has been increased significantly, which is also a critical requirement of the current market.
- Unlike conventional networking, SDN only requires one centralised control plane which offsets the cost of the forwarding plane.
- VM migration becomes easier.
- Automating the configuration is possible.
- Quality of Service can be provided in a more efficient way.

As shown in Figure 2, the SDN controller provides a programmable interface to the OF switches. With the help of this interface, different network applications can be written to control, manage and offer new functionalities. A recent study of several OpenFlow implementations, conducted on a large emulated network with 100,000 hosts and 256 switches, revealed that all the controllers were able to handle about 50,000 new flow requests per second. Besides, according to research going on at Stanford University, the new

architecture will support around 20 million requests per second with about 5,000 switches. Multiple controllers can be used for scalability purposes; these also allow backup controllers to overcome failures and provide almost 100 per cent uptime.

Configuring an existing basic controller (POX controller) in Python

POX is an open source development platform for Python-based software defined networking (SDN) control applications, such as OpenFlow SDN controllers. POX, which enables rapid development and prototyping, is now being more commonly used than NOX, a sister project which is in C/C++.

In this article, we'll configure a basic SDN controller (POX) in Python, and emulate using an open source emulator called Mininet, which provides the functionality to create and operate a virtual network and control it using a controller. To set up the environment for a Mininet emulator, follow the steps described in the article that appeared in the September 2015 issue of *OSFY*. Quick commands to run on the terminal are given below for installation purposes (these commands have been tested on Ubuntu 14.04 and might change for other versions or distributions):

```
sudo apt-get install mininet
```

For installation from the source, type:

```
git clone git://github.com/mininet/mininet
cd mininet
mininet/util/install.sh [options]
```

...where options include the installation of various packages along with Mininet, which can be seen using the *Help* option.

After the successful installation of Mininet (along with the POX controller), we can start POX using the following command:

```
./pox.py log.level -DEBUG misc.of_tutorial
```

The above command tells the POX controller to enable verbose logging and to start the *of_tutorial* file, which acts as an Ethernet hub right now.

Now, start the *Mininet openflow* tool to perform experiments using the following command line:

```
sudo mn -topo single, 3 --mac --switch ovsk --controller
```